



UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH

Facultat d'Informàtica de Barcelona



IoTHarvest



Anna Vincken, David Franco, Marc De Rialp, Pau Quílez, Ermias Valls

Proyecto Tecnologías de la Información

Facultad de Informática de Barcelona (FIB)

Universidad Politécnica de Cataluña (UPC) - BarcelonaTech

30/05/2025

Índice

1. Introducción.....	2
2. Sistema de sensores.....	4
2.1 Hardware utilizado.....	4
2.2 Frecuencia de lecturas.....	5
2.3 Resultado instalación.....	6
2.4 Problemas encontrados.....	7
3. Servidor Web.....	9
3.1 Tecnologías y Arquitectura.....	9
3.2 Flujos de trabajo con el servidor.....	11
4. Base de Datos.....	15
4.1. Colecciones.....	15
5. Frontend.....	20
6. Modelo.....	24
7. VM de la FIB.....	29
8. Dockerizacion de la aplicación.....	30
9. Potencial.....	31
9.1 Hardware huerto.....	31
9.2 Servidor web.....	32
9.3 MongoDB.....	32
9.4 Frontend.....	33
9.5 Modelo IA.....	33
9.6 Docker Compose a Kubernetes.....	33
10. Código del proyecto.....	35
11. Links de interés.....	36

1. Introducción

La agricultura de precisión surge como una respuesta innovadora a los desafíos actuales del sector agrícola, como el cambio climático, la escasez de agua y la necesidad de optimizar la producción de alimentos de manera sostenible. A través de la integración de tecnologías avanzadas, esta disciplina permite una gestión más eficiente de los cultivos, basada en la recopilación y análisis de datos en tiempo real. En este contexto, IoTHarvest es un proyecto que busca desarrollar un sistema de monitorización inteligente para huertos y cultivos, combinando sensores IoT, comunicación inalámbrica y herramientas de análisis de datos para mejorar la toma de decisiones agrícolas.

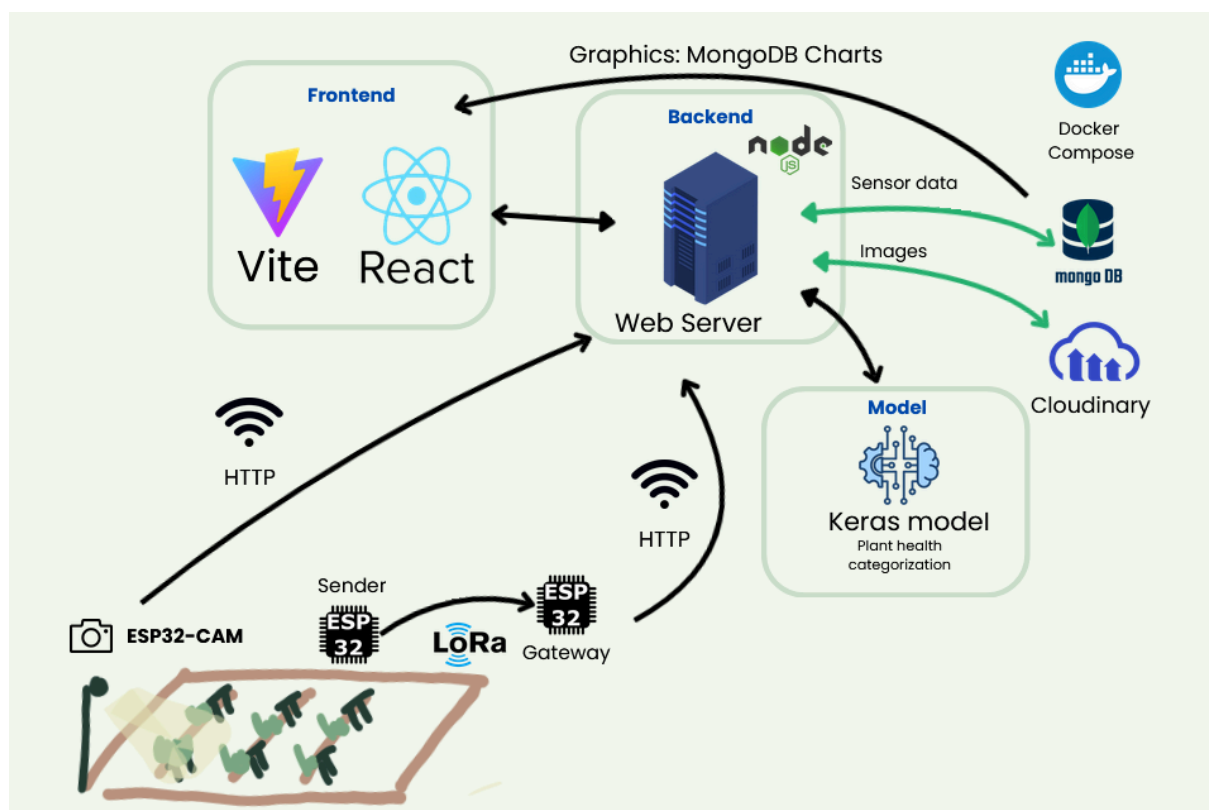
Gracias a su enfoque innovador, IoTHarvest ofrece una solución integral para el monitoreo y gestión inteligente de cultivos, proporcionando a los agricultores una visión completa y detallada del estado de sus tierras. Con acceso a información precisa y en tiempo real, los usuarios pueden tomar decisiones fundamentadas que optimicen el uso del agua, reduzcan desperdicios y maximicen el rendimiento de las cosechas. Esta capacidad de anticiparse a problemas potenciales, como el estrés hídrico o la aparición de plagas, convierte a IoTHarvest en una herramienta clave para una agricultura más eficiente y sostenible.

Además de la supervisión en tiempo real, IoTHarvest transforma los datos recopilados en información valiosa a través de gráficos interactivos y reportes detallados. La plataforma no solo ofrece métricas ambientales clave, sino que también incorpora un sistema de imágenes automatizado que permite evaluar visualmente la evolución de los cultivos. Gracias a esto, los agricultores pueden analizar tendencias, identificar patrones y recibir alertas inteligentes que faciliten la gestión de su producción sin necesidad de estar físicamente en el terreno.

En un mundo donde la digitalización está redefiniendo sectores clave, IoTHarvest representa el futuro de la agricultura conectada. Su diseño modular y escalable permite que cualquier agricultor, desde pequeños productores hasta grandes explotaciones, pueda beneficiarse de esta tecnología sin necesidad de conocimientos técnicos avanzados. Con una plataforma intuitiva y un enfoque basado en la automatización, IoTHarvest simplifica la toma de decisiones y empodera a los agricultores con herramientas de última generación para optimizar su trabajo y garantizar cultivos más sanos y productivos.

Este proyecto no solo responde a las necesidades actuales del sector agrícola, sino que también abre la puerta a un modelo de producción más eficiente y respetuoso con el medio ambiente. Al integrar tecnología de punta en el corazón de la agricultura, IoTHarvest demuestra que la innovación y la sostenibilidad pueden ir de la mano, asegurando un futuro más inteligente y resiliente para la producción de alimentos.

En el siguiente esquema se muestran las distintas tecnologías utilizadas para el desarrollo del proyecto:



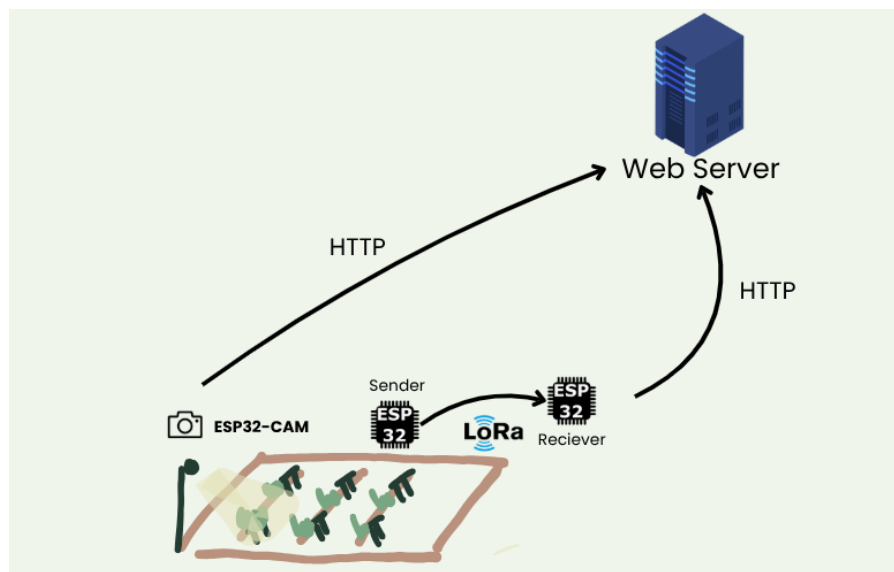
En los siguientes apartados se detallarán cada una de las tecnologías usadas para el correcto funcionamiento del sistema.

2. Sistema de sensores

Para la monitorización del cultivo se ha diseñado un sistema de sensores con el objetivo de poder medir y obtener la siguiente información:

- Humedad del aire
- Temperatura del aire
- Humedad del suelo
- Imagen en tiempo real para evaluar la salud del cultivo (**véase el apartado 6**)

En este apartado se tratará la siguiente sección del esquema de tecnologías:



Es decir, todo lo referente con la obtención de datos del cultivo y la comunicación con el servidor web que gestiona el resto de componentes del proyecto.

2.1 Hardware utilizado

Para realizar las lecturas y enviarlas al servidor web se han utilizado los siguientes componentes:

- TTGO LORA32 (Sender) [1]
- TTGO T-Beam ESP32 (Receiver) [2]
- ESP32-CAM OV2640 [3]
- Capacitive Soil Moisture Sensor [4]
- Sensor DHT11 [5]

La placa TTGO LORA32 es un ESP32 que integra un módulo de LoRa, una pantalla OLED, entre otros componentes. Se ha utilizado para tomar las lecturas de humedad del suelo con el Capacitive Soil Moisture Sensor y las lecturas de humedad y temperatura del aire con el sensor DHT11. Una vez tomadas las lecturas, se utiliza para agrupar toda la información en un objeto .json y generar un paquete LoRa con el objeto .json de contenido. La placa Sender se encarga además de la trama con las lecturas a la placa Receiver a través de LoRa.

La placa Receiver actúa como gateway de la placa Sender. En otras palabras, lo que hace Receiver es desencapsular las lecturas de la trama LoRa y enviarlas al servidor web mediante un cliente http. El servidor tiene un endpoint abierto para que Receiver pueda hacer post de las lecturas que ha recibido de Sender.

Por otro lado, la placa ESP32-CAM OV2640 envía las imágenes que toma del cultivo al servidor web sin tener que utilizar ningún tipo de gateway. Esta placa tiene incorporado un módulo WiFi que le permite enviar las imágenes directamente al endpoint que tiene servidor web para recibir las imágenes.

2.2 Frecuencia de lecturas

El estado de un cultivo no es una variable que experimente cambios significativos en cuestión de segundos o minutos. Por esta razón, se ha estimado que una frecuencia de lectura cada 30 minutos es suficiente para obtener datos representativos del ambiente y del suelo. Esta periodicidad permite un seguimiento adecuado sin generar una sobrecarga innecesaria de datos o consumo energético.

En cuanto a la captación de imágenes mediante la placa ESP32-CAM OV2640, se ha definido una frecuencia de una fotografía por día. Esta imagen diaria permite evaluar visualmente la salud del cultivo, detectar posibles anomalías y complementar los datos obtenidos por los sensores de humedad y temperatura.

Adicionalmente, se ha implementado un sistema de detección de eventos nocivos para el cultivo, como pueden ser temperaturas elevadas o niveles de humedad en el suelo por debajo de los umbrales definidos. En caso de que se detecte alguno de estos eventos, se rellena un campo del objeto .json denominado status, en el cual se asigna un código de error específico en función del tipo de evento encontrado.

Cuando el servidor recibe este objeto con un estado de alerta, se genera automáticamente una notificación por correo electrónico dirigida al usuario. En

dicho correo se le informa sobre la situación actual de su cultivo y se le proporcionan recomendaciones sobre posibles acciones a tomar para mitigar el problema detectado. Este mecanismo permite una respuesta más rápida y eficiente ante condiciones adversas.

2.3 Resultado instalación

Uno de los aspectos más importantes a tener en cuenta al instalar sistemas electrónicos en entornos agrícolas es la protección frente a la humedad, la lluvia y otros factores ambientales. Dado que las placas estarán expuestas directamente en el huerto, es fundamental impermeabilizarlas correctamente para evitar daños por agua, oxidación de componentes o cortocircuitos.

Para garantizar la seguridad y durabilidad del sistema, se diseñó una carcasa a medida para la placa ESP32-CAM, la cual fue impresa en 3D utilizando material plástico resistente a la intemperie. Esta carcasa protege tanto la cámara como la electrónica interna, sin obstruir el campo de visión, permitiendo su uso continuo en el exterior.



Por otra parte, la placa TTGO Sender fue introducida en un recipiente plástico estanco, lo que permite su instalación en el campo sin riesgo de filtraciones o fallos eléctricos. Estas medidas aseguran un funcionamiento estable y continuo del sistema incluso bajo condiciones ambientales adversas.

Con el objetivo de validar el funcionamiento del sistema en un entorno real, se ha realizado la instalación de todos los componentes en un cultivo de pruebas. Esta fase ha permitido comprobar la eficacia de la recolección de datos, la transmisión mediante LoRa y el envío de información al servidor web, así como la correcta integración de la cámara ESP32-CAM para la captura y envío de imágenes.

A continuación, se presentan varias imágenes que documentan el proceso de instalación y el estado final del sistema desplegado en el campo. En ellas se puede observar la disposición de los sensores, las placas electrónicas y la cámara, así como su integración con el entorno natural del cultivo.



2.4 Problemas encontrados

Durante la implementación y validación del sistema en el cultivo de pruebas, surgieron diversos imprevistos que permitieron identificar puntos críticos y aplicar soluciones para mejorar la robustez del proyecto.

En primer lugar, el cultivo inicial fue afectado por una plaga que comprometió completamente las plantas, haciendo inviable continuar con las pruebas en ese entorno. Como consecuencia, fue necesario replantar el cultivo y reiniciar el proceso de instalación y seguimiento.

Otro de los problemas detectados fue que la toma de lecturas a intervalos regulares provocaba picos de tensión en las placas, especialmente en la TTGO LORA32 (Sender), lo que ocasiona la aparición de valores nulos o incoherentes en las lecturas de los sensores. Para mitigar este problema, se incorporó un sistema de detección de errores en tiempo real: si se detecta una lectura fuera de los rangos esperados o vacía, se repite automáticamente la lectura hasta obtener un valor razonable. Esta solución mejora la confiabilidad de los datos enviados al servidor.

Además, durante el diseño del sistema se consideró la posibilidad de que el terreno en el que se desee instalar el sistema no cuente con acceso a una red WiFi o

conexión a internet fija. Para hacer frente a esta limitación, el sistema se diseñó para manejar un flujo pequeño de datos y así ser compatible con tecnologías móviles como 4G y 5G, lo cual permite utilizar una tarjeta SIM instalada en un módulo compatible para establecer la conexión a internet de forma autónoma. De esta manera, el sistema puede operar en zonas rurales o remotas sin depender de infraestructuras externas de conectividad.

Estos contratiempos han contribuido significativamente a reforzar el diseño general del sistema, haciéndolo más adaptable, resiliente y apto para entornos reales de uso agrícola.

3. Servidor Web

El servidor web representa el núcleo del sistema IoT Harvest, actuando como el punto central de comunicación entre todos los componentes del proyecto. Su arquitectura está diseñada para gestionar de manera eficiente la recepción, procesamiento y almacenamiento de datos provenientes de múltiples fuentes, mientras proporciona una API robusta para el frontend y mantiene la integridad de los datos a través de múltiples sistemas de almacenamiento.

3.1 Tecnologías y Arquitectura

El servidor de IoT Harvest está desarrollado en Node.js con Express.js, ejecutándose en el puerto 8080. Utiliza Mongoose como ODM para interactuar con MongoDB, optimizada para almacenar datos en series temporales.

La gestión de archivos, como las imágenes enviadas por las cámaras ESP32, se realiza con Multer, que permite subir archivos con validaciones y almacenamiento temporal. Las imágenes se almacenan localmente, en caché y en un directorio para análisis con IA, además de subirse a Cloudinary de forma controlada.

La autenticación se basa en JWT y bcrypt, ofreciendo sesiones seguras y protección de credenciales. Además, Nodemailer se utiliza para enviar notificaciones por correo electrónico basadas en eventos críticos detectados por los sensores.

El sistema permite comunicación segura entre el frontend y el backend mediante CORS, y todas las comunicaciones usan el formato JSON.

Gestión de Archivos Multimedia

El sistema emplea Multer como middleware para la gestión de archivos enviados mediante HTTP. Este componente permite recibir imágenes directamente desde las cámaras ESP32, almacenarlas temporalmente en el servidor y validar que cumplan con el formato permitido, en nuestro caso (únicamente archivos JPG o JPEG). Además, facilita la configuración de rutas de almacenamiento y la generación automática de nombres únicos, utilizando timestamps ajustados a la zona horaria española.

Por otra parte, Cloudinary se integra como solución externa de almacenamiento en la nube. Una vez que Multer ha procesado y validado la imagen, el backend utiliza la API de Cloudinary para subirla al entorno cloud. Esta plataforma ofrece

almacenamiento seguro, URLs públicas para acceso remoto, y herramientas avanzadas como optimización o transformación de imágenes bajo demanda. La integración se realiza mediante el SDK oficial de Cloudinary para Node.js, utilizando credenciales protegidas mediante variables de entorno.

Seguridad y Autenticación

El backend implementa un sistema de autenticación utilizando JSON Web Tokens (JWT) y bcrypt para garantizar la seguridad de las credenciales y las sesiones.

Al registrarse, las contraseñas de los usuarios se cifran con bcrypt antes de almacenarse en la base de datos.

Durante el inicio de sesión, el servidor compara la contraseña introducida con el hash almacenado y, si coincide, genera un JWT firmado que se envía al cliente. Este token, con una validez limitada (por ejemplo, 24 horas), debe incluirse en las cabeceras de las peticiones a rutas protegidas, permitiendo sesiones seguras sin necesidad de almacenar contraseñas en el cliente ni mantener sesiones persistentes en el servidor.

Notificaciones y Correos Electrónicos

El sistema integra un mecanismo de notificaciones mediante correos electrónicos automáticos utilizando Nodemailer. Cuando el servidor detecta eventos críticos a partir de los datos de los sensores como batería baja, humedad insuficiente o temperatura fuera de rango, etc. Se activa el envío de un correo al usuario registrado. Para ello, se utilizan plantillas HTML personalizadas adaptadas a cada tipo de alerta.

Comunicación Frontend-Backend

La comunicación entre el frontend (puerto 8081) y el backend está protegida mediante la configuración de CORS, permitiendo únicamente el acceso desde dominios autorizados.

La API expone endpoints para:

- Iniciar sesión.
- Crear cuenta
- Consultar los datos de los sensores.
- Consultar las imágenes locales
- Descargar imágenes de Cloudinary
- Obtener resultados e interpretaciones del modelo de IA.

Todas las respuestas se devuelven en formato JSON, y el sistema convierte automáticamente las fechas al horario de España para coherencia con el usuario final.

3.2 Flujos de trabajo con el servidor

Procesamiento de Datos de Sensores

El dispositivo ESP32 envía periódicamente datos al servidor backend mediante un endpoint HTTP POST (/api/sensores). Los datos incluyen temperatura, humedad del aire, humedad del suelo, nivel de batería y un código de estado que puede señalar situaciones de alerta (como baja humedad o batería baja).

1. Recepción y Validación

Al recibir la petición, el backend extrae los valores del cuerpo de la solicitud. Si la cabecera especial x-test-data está presente y su valor es "true", los datos se marcan como de prueba (isTestData: true). Esto permite distinguir entre datos reales y datos generados durante simulaciones a través de los scripts de testeo.

2. Campos añadidos

El servidor añade automáticamente los siguientes campos:

- **timeServer**: Marca temporal generada por el servidor en el momento exacto de la recepción.
- **isTestData**: Indica si los datos son reales o de prueba.

3. Almacenamiento y Caché

Una vez validados, los datos se procesan del siguiente modo:

- Se actualiza la caché en memoria para permitir acceso inmediato desde el frontend.
- Se escriben en un archivo JSON local (histórico diario) como copia redundante.
- Se almacenan en MongoDB, utilizando una colección optimizada para series temporales.

4. Gestión de Alertas

Si el campo status recibido es distinto de 0, se considera que hay una alerta. En ese caso, el servidor genera y envía automáticamente un correo de aviso usando la dirección especificada al loguearse.

Campos Procesados

- **temperatura**: Valor numérico de temperatura ambiente.
- **humedad_aire**: Porcentaje de humedad relativa del aire.
- **humedad_suelo**: Porcentaje de humedad en el suelo.
- **bateria**: Nivel de batería del dispositivo (opcional).
- **status**: Código de estado que puede indicar alertas.
- **timeServer**: Fecha y hora de recepción en el servidor.
- **isTestData**: Indica si los datos son reales o de prueba.

Procesamiento de Imágenes

Cada vez que el servidor recibe una imagen, se inicia un flujo de procesamiento diseñado para garantizar su disponibilidad inmediata, almacenamiento a largo plazo y accesibilidad desde múltiples componentes del sistema. El proceso sigue los siguientes pasos:

1. Nombrado Temporal

La imagen se renombra automáticamente utilizando un timestamp con el formato **YYYY-MM-DD-HH-MM-SS**, ajustado a la zona horaria de España. Esto asegura una identificación única, orden cronológico claro y evita conflictos por nombres duplicados.

2. Almacenamiento en Múltiples Ubicaciones

La imagen se distribuye simultáneamente en cuatro ubicaciones:

- **a) Caché Local para el Frontend:**
Se guarda una copia en `data/cache/imageCache/`, donde se mantienen únicamente las últimas 14 imágenes. Esta caché permite al frontend mostrar de forma inmediata una galería actualizada. El sistema elimina automáticamente las imágenes más antiguas cuando se supera el límite.
- **b) Almacenamiento Permanente:**
Se realiza una copia de seguridad en `data/images/`. Aquí se almacenan todas las imágenes recibidas sin límite, lo que permite mantener un histórico completo y consultable para auditorías o análisis posteriores.
- **c) Directorio para el Modelo de IA:**
La imagen se copia a `data/leaf_classification/last_image/`, el directorio que monitoriza el modelo de predicción. Antes de guardar la nueva imagen, se elimina cualquier imagen anterior, asegurando que el modelo siempre procesa la imagen más reciente.
- **d) Almacenamiento en la Nube (Cloudinary):**
La imagen también se sube a Cloudinary, lo que permite su acceso desde cualquier ubicación y su integración con servicios externos. Para evitar saturar la red o llenar el almacenamiento en la nube por errores o frecuencia excesiva, se ha implementado un mecanismo de *cooldown* configurable que limita la frecuencia de subida.

Integración con Inteligencia Artificial

El sistema incorpora un modelo de inteligencia artificial para analizar imágenes de las hojas y determinar su estado de salud.

```
data/leaf_classification/  
├─ last_image/  
├─ predicted_images/  
├─ logs/  
└─ last_prediction.txt
```

La carpeta `last_image/` contiene siempre solo la imagen más reciente que debe procesar el modelo, eliminando las anteriores para evitar reprocesos. Las imágenes clasificadas se almacenan en `predicted_images/` para mantener un histórico organizado.

En la carpeta `logs/` se guardan todos los resultados de cada clasificación realizada. Por otro lado, el archivo `last_prediction.txt` contiene únicamente el resultado más reciente.

Un script de monitorización usa `inotifywait` para vigilar en tiempo real la carpeta `last_image/`. Al detectar una nueva imagen, ejecuta automáticamente el script de predicción, que procesa la imagen con el modelo y actualiza `last_prediction.txt`. El backend expone estos resultados mediante un endpoint GET.

4. Base de Datos

Para este proyecto se ha propuesto implementar la base de datos con MongoDB. Es esencial que para nuestro sistema se disponga de una base de datos capaz de gestionar grandes volúmenes de información temporal, con flexibilidad y eficiencia, y MongoDB cumple con estos requisitos.

MongoDB es una base de datos no relacional que permite almacenar la información en formato JSON, facilitando una estructura de datos flexible y adaptable. Esta flexibilidad es muy relevante en proyectos IoT, ya que la cantidad de datos que se pueden recibir en un corto periodo de tiempo puede ser realmente elevado. Además, facilita mucho el proceso de recepción de datos desde los sensores ya que estos se envían en formato JSON también.

Un aspecto que hace muy atractivo a MongoDB es la capacidad que tiene de gestionar datos temporales mediante colecciones Time Series. Esta funcionalidad está optimizada para registrar información que se guarda de manera continua y periódica. Las colecciones Time Series permiten una mejor compresión de los datos, una reducción en el uso del espacio y un mayor rendimiento en consultas basadas en rangos temporales.

Además, MongoDB facilita notablemente la integración con MongoDB Charts, una herramienta usada en el proyecto para la visualización directa de datos almacenados, permitiendo generar gráficos sobre el estado del cultivo.

4.1. Colecciones

Una colección es una agrupación de documentos relacionados que se almacenan dentro de una base de datos, de manera parecida a una tabla en una base de datos relacional, pero con una estructura más flexible. Durante la implementación del proyecto se han llegado a implementar un total de cuatro colecciones.

Users:

Esta colección es la que almacena los usuarios que se van registrando a nuestra web. Se almacena su nombre de usuario y su contraseña encriptada.

```
_id: ObjectId('68207d5460e7f66390b5289f')
username: "ermiasvalls@gmail.com"
password: "$2b$12$i31gAqC6jdaqaJWCynx5p.4EATEMcTBjNrLF0qtnp/Go2T9MGVPZO"
__v: 0

_id: ObjectId('68207e5760e7f66390b528a7')
username: "iotharvest.pti@gmail.com"
password: "$2b$12$xLFPTMstvyOPwlcqcVtR/eKwswUZt0SknnXQ3Gox55FvKXFBc8biq"
__v: 0

_id: ObjectId('68208016c85fb4fb468d6f4a')
username: "a@gmail.com"
password: "$2b$12$njOP896WhV3ydW1k7ThmcuxtbQPkoSEWaTwh/wK94gYwyrBBGYqgu"
__v: 0

_id: ObjectId('6820814fd477a620f76cc389')
username: "b@gmail.com"
password: "$2b$12$skgVmRUV.0RQr0nhb9w.efsdWtWrAD04GrKKTudF9C5Km0AwmX.Qm"
__v: 0

_id: ObjectId('6835786232e03923003984aa')
username: "sergio.ortiz.olivares@estudiantat.upc.edu"
password: "$2b$12$Xk0fFAaNA/ymdVJcuJ4MjeTo0jQsJNqwI4w0x3Yg.nhEfuk0tIEpC"
__v: 0
```

ESP32backup:

Esta colección ha servido única y exclusivamente para disponer de datos de backup, en el supuesto caso que durante la implementación del proyecto se produjeran pérdidas de datos en otras colecciones. Esta colección no está pensada para el usuario final.

SensorsData:

Era la colección que, inicialmente, guardaba los datos recibidos de los sensores. Esta colección no estaba implementada con Time Series, por lo que no se aprovechaba al máximo la eficiencia y flexibilidad que MongoDB ofrece en proyectos de este ámbito. Actualmente esta colección ya se encuentra vacía.

SensorsDataOptimized:

La colección más interesante de todas. Es la definitiva para el almacenamiento de los datos que se reciben de los sensores. Esta colección ya está creada con Time Series [6].

Los atributos que contiene esta colección son:

- **timeServer**: indica el momento temporal de la recepción de la información.
- **humedad_aire**, **temperatura** y **humedad_suelo**: son los parámetros realmente necesarios para el usuario de cara al monitoreo de su cultivo.
- **status**: es un indicador que va del 0 al 7, indicando si hay algún contratiempo con el cultivo. En caso de que el valor sea 0, el cultivo no está en peligro. Los demás números indican los posibles casos peligrosos que se podrían contemplar en el cultivo.
- **isTestData**: es un atributo dedicado a los desarrolladores, de manera que se puede identificar si el dato recibido proviene de los sensores, lo que indica que es un dato útil, o si es un dato recibido desde un script única y exclusivamente para testear el correcto funcionamiento de la recepción de datos. Si la información proviene de un sensor, este atributo vale false, en caso contrario, vale true.

En esta imagen se puede apreciar como se visualiza un dato de esta colección desde mongoDB Compass.

```
timeServer : 2025-05-25T10:01:03.165+00:00
humedad_aire : 57.8
status : 0
_id: ObjectId('6832ea5fe13126c5396da835')
__v : 0
temperatura : 23.9
isTestData : false
humedad_suelo : 3092
```

Como se ha mencionado anteriormente, en esta colección ya se implementa Time Series. A diferencia de la colección sensorsData, la base de datos trabaja con la información recibida de una forma que optimiza considerablemente el espacio que dispone la base de datos. Para ello, agrupa la recepción de datos en distintos bloques, conocidos como “buckets”. El criterio para agruparlos viene dado por tres campos principales en Time Series:

- timeField
- metaField
- granularity

timeField guarda el momento temporal en el que se ha recibido un documento en la colección, es decir, un dato del sensor. Por lo tanto, en este caso, timeField se asocia con timeServer.

metaField es un campo opcional donde se le debería asociar un atributo que sea prácticamente estático como, por ejemplo, el identificador de un huerto. Como este proyecto se ha basado en un único cultivo, este campo está vacío y no se usa, por lo que por ahora no se comentará nada más sobre este campo.

granularity hace saber a la colección la frecuencia con la que se reciben datos. Este campo no corresponde a ningún atributo, sino que el desarrollador lo indica al crear la colección. Se puede indicar una frecuencia de segundos, minutos u horas. En el caso de este proyecto la granularidad es de minutos, ya que se envían datos cada media hora.

Para configurar los buckets, se hace uso del timeField y granularity, de manera que se realiza un cálculo que lo hace la propia configuración Time Series, y se establece el rango temporal de ese bucket. En otras palabras, se establece el máximo intervalo de tiempo que puede contener un bucket.

Como configuración adicional, se puede configurar que ciertos datos que ya no tengan efecto en el interés del usuario, como pueden ser datos antiguos, se eliminen de la colección. Esto se hace activando el campo `expireAfterSeconds`, indicando la cantidad de segundos que deben transcurrir para que los datos que cumplan esa longevidad temporal se eliminen de la colección, permitiendo ganar espacio en la base de datos. En este proyecto se ha decidido que los datos expirarán cuando transcurran aproximadamente 15 días.

La configuración de la colección `sensorsDataOptimized` se puede observar en esta imagen:

```
{
  name: 'sensorsDataOptimized',
  type: 'timeseries',
  options: {
    expireAfterSeconds: Long('1295998'),
    timeseries: {
      timeField: 'timeServer',
      granularity: 'minutes',
      bucketMaxSpanSeconds: 86400
    }
  },
  info: { readOnly: false }
}
```

Se puede observar que el campo `bucketMaxSpanSeconds` es el resultado del cálculo realizado con el `timeField` y `granularity`, dando como resultado que el máximo intervalo de tiempo del bucket es de 86400 segundos, lo que equivale a un día.

5. Frontend

El objetivo principal del frontend es permitir al usuario consultar de forma clara y sencilla los datos de monitorización del huerto. Para ello, se ha utilizado React junto con Vite como entorno de desarrollo, con el fin de crear una interfaz moderna, rápida y eficiente.

React es una biblioteca de JavaScript desarrollada por Meta, que permite construir interfaces de usuario de forma eficiente mediante componentes reutilizables y reactivos, lo que facilita la creación de aplicaciones web dinámicas y modulares. Vite, por su parte, es una herramienta de construcción moderna que se utiliza junto con React para acelerar el desarrollo, ya que ofrece recarga en caliente, tiempos de arranque casi instantáneos y una experiencia de desarrollo mucho más fluida en comparación con herramientas tradicionales.

La aplicación web está compuesta por tres pantallas principales: Login, Sign Up y Dashboard. Las dos primeras permiten gestionar el acceso de usuarios, mientras que el Dashboard muestra todos los datos recogidos por los sensores y la cámara.

Dashboard:

La pantalla de Dashboard ha sido diseñada con un enfoque minimalista, priorizando la accesibilidad y la claridad visual de los datos. En esta vista se muestran:

Gráficas de:

- Humedad del aire
- Humedad del suelo
- Temperatura

Estas gráficas se han hecho a partir de MongoDB Charts, una herramienta nativa de MongoDB Atlas que permite crear gráficos de manera muy intuitiva para las colecciones que lo necesiten.

Hemos decidido agrupar los datos de manera que se vean las últimas horas de evolución para las respectivas medidas. Se calcula la media y se muestra como un punto por hora. Para añadirlos al Frontend simplemente con un embed permite la visualización directa, si permitimos el acceso no autenticado en MongoDB Charts.

Datos en tiempo real obtenidos desde la misma base de datos, que muestran valores actuales como:

- Temperatura
- Humedad ambiental
- Humedad del suelo
- Hora de la última actualización

Imagen en directo del huerto con una pequeña descripción generada por una IA de predicción del estado del cultivo, que analiza la salud de la planta a partir de la imagen.

Predicción meteorológica proporcionada por la API de OpenMeteo, que se muestra en forma de iconos y valores para los próximos días, facilitando así la planificación de riegos o cuidados especiales, también se incluyen las temperaturas máximas y mínimas de cada día.

Además, desde el frontend se realizan peticiones a distintos endpoints del backend para obtener y mostrar la información necesaria en el Dashboard. En concreto, se accede a un endpoint para obtener los datos actuales del huerto (temperatura, humedad del aire y del suelo), otro para recuperar los datos históricos que se representan en las gráficas, uno adicional que proporciona la imagen más reciente del cultivo, y finalmente un endpoint que devuelve la predicción generada por la IA sobre el estado de la planta.

Tecnologías utilizadas

React: Biblioteca principal para construir la interfaz de usuario con componentes reutilizables y estado reactivo.

Vite: Herramienta de desarrollo que mejora los tiempos de carga y permite una experiencia de desarrollo más ágil.

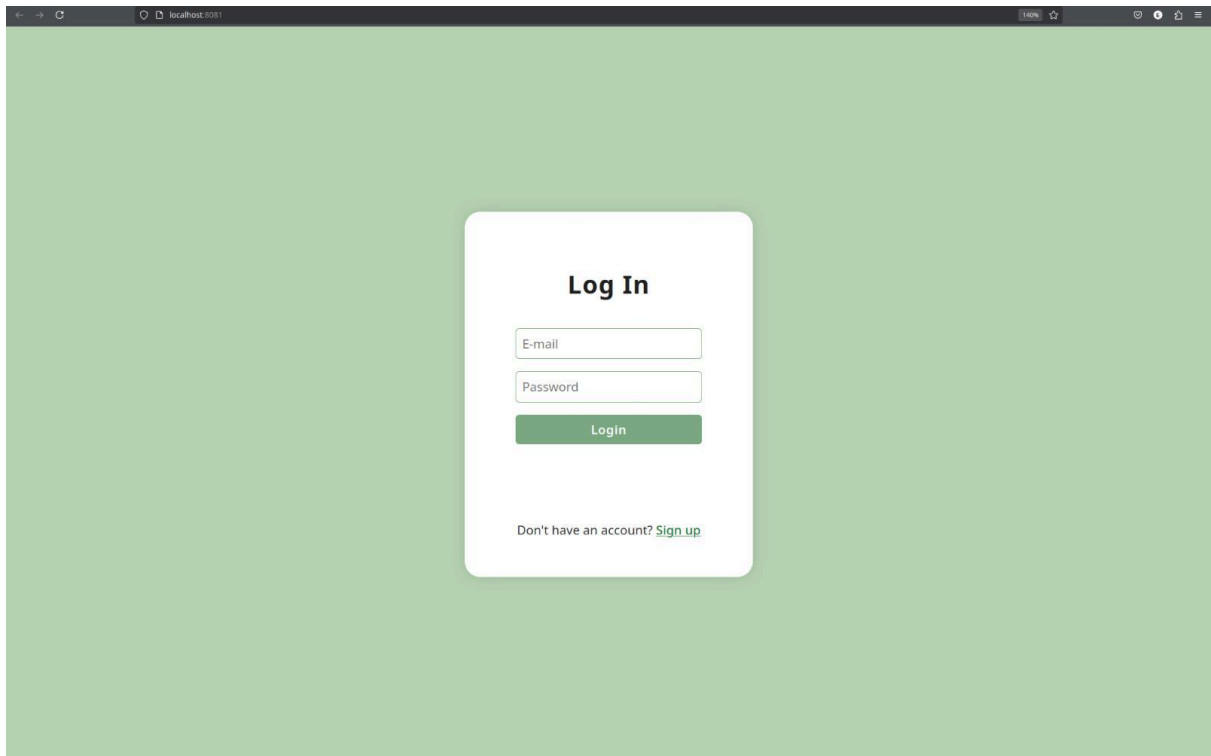
OpenMeteo API: Servicio de meteorología que usamos para obtener la predicción del tiempo.

Inteligencia Artificial: Implementación de un modelo que analiza las imágenes del cultivo para ofrecer una valoración automática del estado del huerto.

Diseño de interfaz

Hemos optado por un diseño limpio y minimalista, con el objetivo de que el usuario pueda acceder de forma rápida y sencilla a los datos más relevantes sobre el estado del huerto. Las gráficas y lecturas se presentan en un formato visualmente accesible, evitando sobrecargar la interfaz.

A continuación, se adjuntan capturas de pantalla de las vistas Login, Sign Up y Dashboard



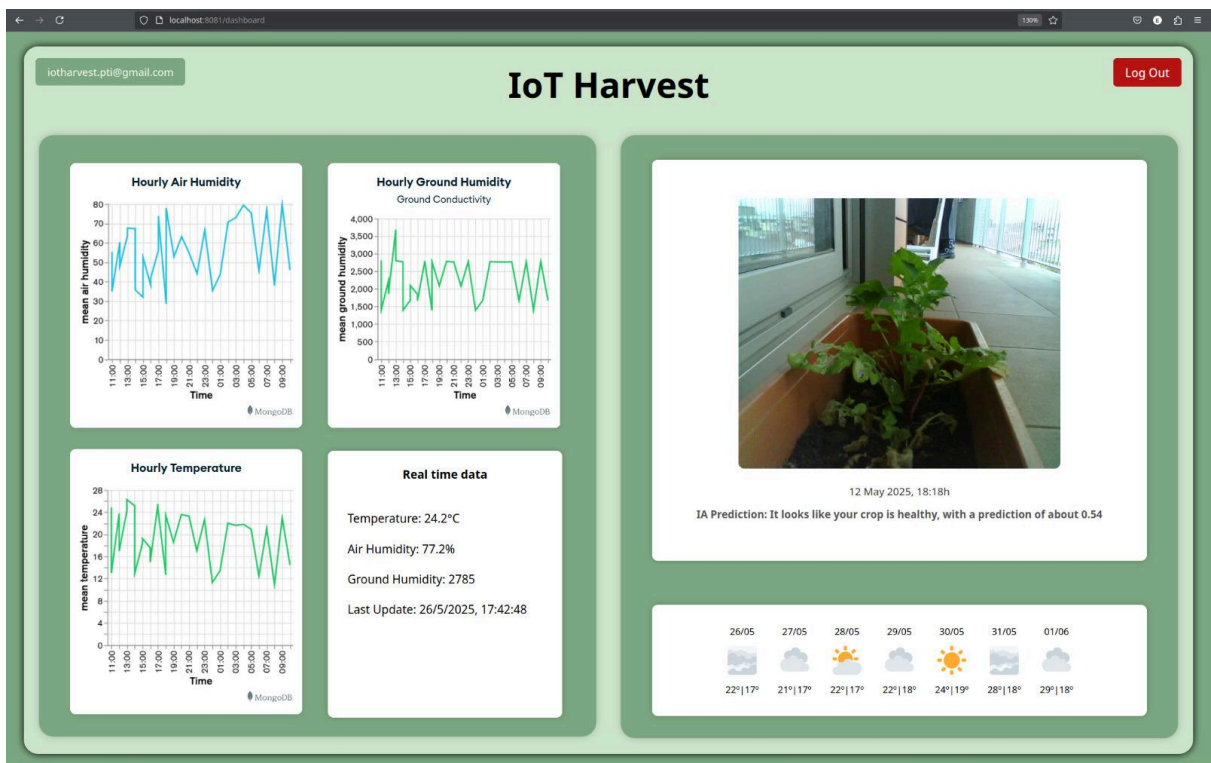
localhost:5051/create-account

145%

Sign up

Sign up

Have an account? [Log in](#)



6. Modelo

Hemos visto que hoy en día la IA está en auge y nos parecía realista implementar una detección de salud de la planta. Una vez entramos a investigar, las opciones no eran tantas como parecía, por lo menos sin coste, y decidimos entrenar un modelo ya establecido, con un dataset que encontrásemos.

Este componente del proyecto consistió en diseñar y entrenar un modelo de red neuronal convolucional (CNN) con Keras, capaz de clasificar imágenes de hojas de rábano como saludables o no saludables. Explicaré un poco las fases que ha seguido este modelo y como se ha acabado ajustando.

Enlace dataset utilizado[7]. Solo hemos seleccionado las "Fresh" para healthy y "Downy mildew" para unhealthy ya que no queríamos ampliar mucho y simplemente hacer una clasificación binaria.

No os enviamos el dataset, ni aparece en el repositorio ya que ocupa mucho espacio y todos los entrenamientos se han hecho en local, así que en la memoria mostraré un par de imágenes para entender cómo se ha hecho.

Ejemplos imágenes del dataset:



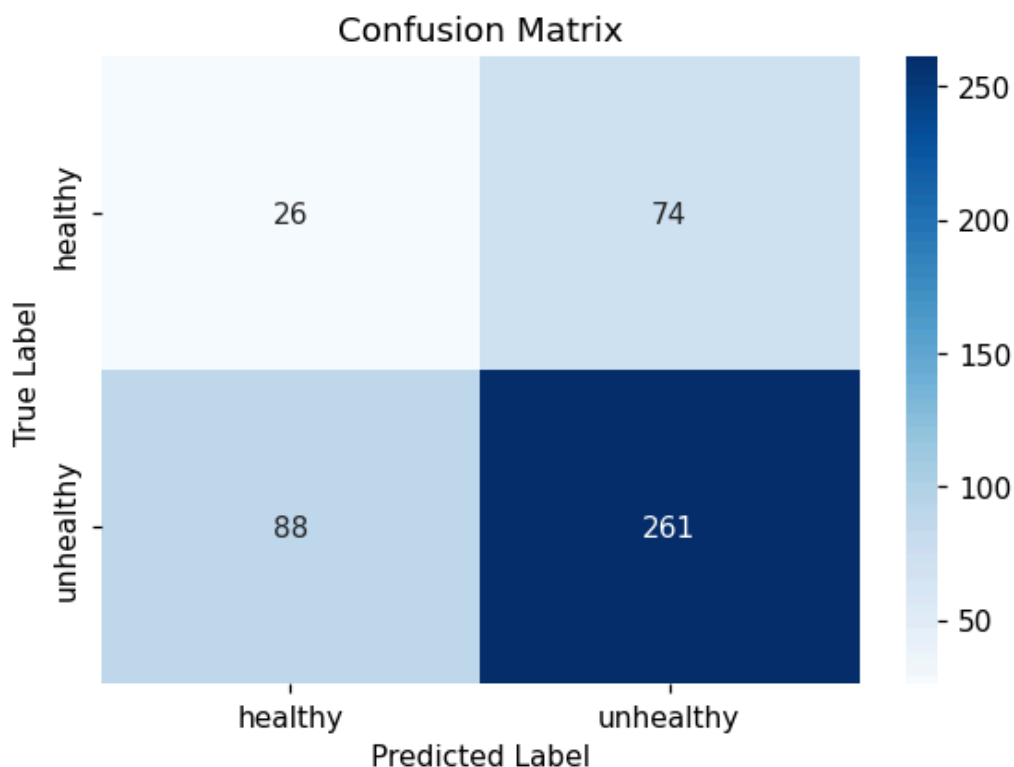
"Fresh" o healthy	"Downey mildey" o unhealthy (moho)
-------------------	------------------------------------

Fase inicial: Pruebas con EfficientNetB0

En una primera etapa, utilicé como base el modelo preentrenado EfficientNetB0 con pesos de ImageNet, con la intención de beneficiarnos de su capacidad de transferencia de aprendizaje. Sin embargo, el modelo presentaba un fuerte sesgo hacia la clase healthy, clasificando la mayoría de las imágenes en esa categoría.

Para evaluar el rendimiento, incorporé al final del proceso una matriz de confusión. Idealmente, los valores deberían concentrarse en la diagonal (es decir, clasificaciones correctas), pero en este primer intento el sesgo era claro.

Para intentar arreglar esto, inocentemente, intenté ajustar el umbral de decisión del modelo (de 0.5 a un valor inferior), pero esto solo invirtió el problema, clasificando ahora en su mayoría como unhealthy.



Ajustes intermedios: Fine-tuning y focal loss

Para mejorar la capacidad de discriminación del modelo, opté por desbloquear más capas del modelo base y realizar fine-tuning. También sustituí la función de pérdida por focal loss, que penaliza más los errores en clases minoritarias y es útil en casos de desbalance o clasificación difícil.

No voy a entrar mucho en detalle de los métodos de clasificación, pero si interesa en el enlace [8] del anexo, entra en más detalle.

para el artículo “Focal Loss vs. Binary Cross Entropy Loss”, explica un poco para que sirve cada método.

Cambio de Enfoque: Clasificación por textura

Durante la experimentación, detecté que el modelo tomaba como características clave la forma y orientación de las hojas, lo cual no era deseable. Nuestro objetivo era que se centrara en la textura, que es más relevante para detectar signos de enfermedad.

Para forzar al modelo a fijarse en la textura:

- Se redujo el dataset para balancear las clases (200 imágenes por clase).
- Implementé un sistema que extrae un parche central de 224 x 224 píxeles de cada imagen, eliminando así el sesgo por forma. Está hecho de esta manera porque la mayoría de imágenes del dataset están bien centradas.

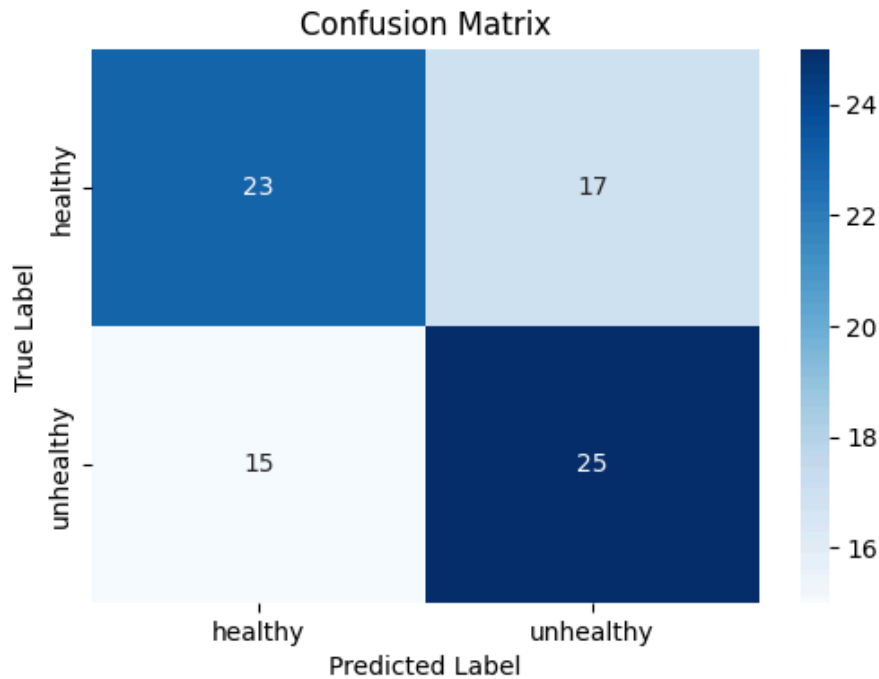
	
Parche de hoja “healthy”	Parche de hoja “unhealthy”

- Sostituí EfficientNetB0 por DenseNet121, modelo que favorece la reutilización de características, lo cual se adapta bien al análisis de patrones

de textura: Según la IA de Chat GPT, DenseNet121 era la más apropiada para esto, y investigando algunos artículos como en el anexo, enlace [9], explican con bastante detalle su funcionamiento.

Resultados Finales

El nuevo modelo fue entrenado con esta configuración, y la evaluación final sobre el conjunto de validación produjo una matriz de confusión mucho más equilibrada:



Esto indica que el modelo ha aprendido a distinguir de forma más eficaz entre hojas sanas y enfermas. El umbral se ajustó finalmente a 0.5 de nuevo. No es el modelo que mejor funciona pero apañamos.

Despliegue y automatización

Para su integración en el sistema final, el modelo se desplegó en el servidor. Un pequeño script en segundo plano monitoriza la carpeta donde llegan nuevas imágenes:

```
Shell

while true; do

    inotifywait -e create backend/data/leaf_classification/last_image/. &&
    python3 model/predict_mv.py

done
```

Cada vez que se detecta una nueva imagen:

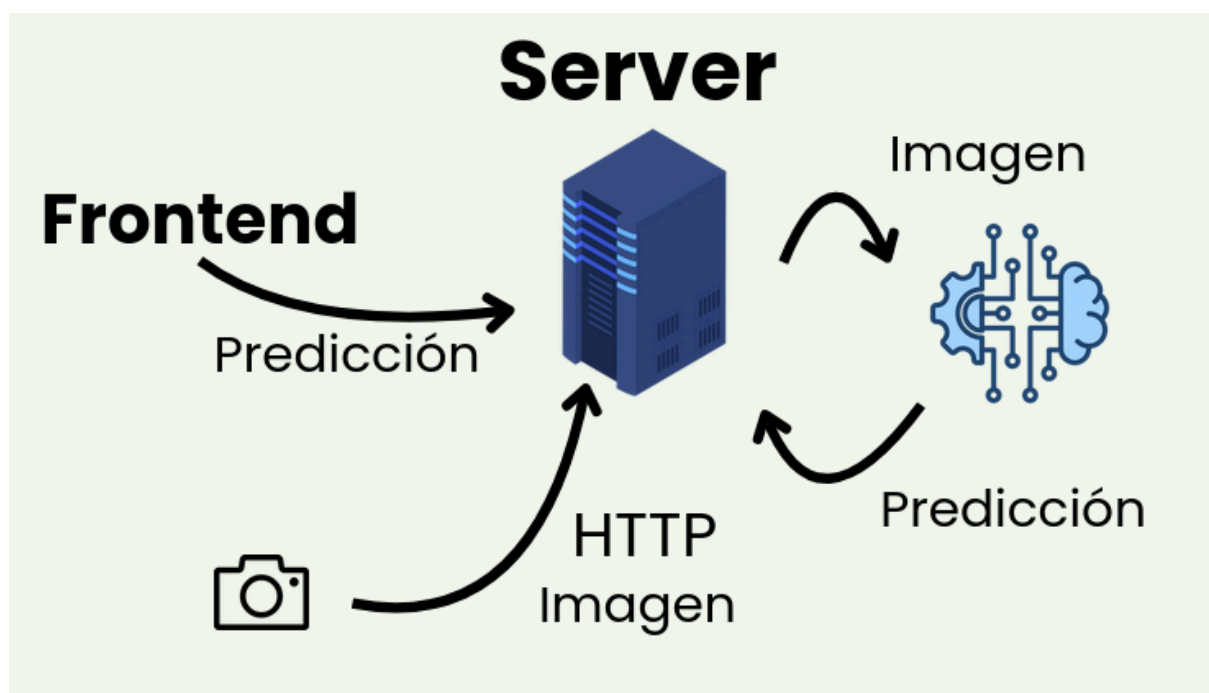
- Se procesa extrayendo un parche central.
- Se normaliza y pasa por el modelo.
- Si la predicción es ≥ 0.5 se clasifica como healthy; si < 0.5 como unhealthy.

El resultado se escribe en un archivo de texto (last_prediction.txt), que luego el servidor utiliza para responder al frontend.

Fragmento relevante del script predict_mv.py:

```
94     if pred_value < 0.5:  
95         result = f"It looks like your crop is unhealthy, with a prediction of about {pred_value:.2f}"  
96         exit_code = 2  
97     else:  
98         result = f"It looks like your crop is healthy, with a prediction of about {pred_value:.2f}"  
99         exit_code = 1  
100
```

Este flujo permite un sistema de predicción automático, liviano y era de las maneras más sencillas de conectarlo con el servidor.



Funcionamiento predicción de IA en el servidor

7. VM de la FIB

Una vez tuvimos una estructura funcional y estable del sistema IoTHarvest, desplegamos toda la aplicación en una máquina virtual proporcionada por la FIB, gestionada mediante la plataforma OpenNebula.

En esta VM instalamos todas las dependencias necesarias: entornos de ejecución como Node.js y Python, bases de datos como MongoDB, librerías específicas para el modelo de inteligencia artificial, herramientas de red, servicios de correo y utilidades como inotify-tools, entre otras.

Para automatizar la ejecución del sistema, creamos un servicio de systemd que se activa automáticamente al arrancar la máquina. Este servicio está ubicado en el archivo: `/etc/systemd/system/multi-user.target.wants/iotharvest.service`

```
[Unit]
Description=Iniciar servidor IoTHarvest
After=network.target

[Service]
Type=simple
ExecStart=/home/alumne/IoTHarvest/start_server.sh
User=alumne
WorkingDirectory=/home/alumne/IoTHarvest
Restart=on-failure

[Install]
WantedBy=multi-user.target
```

Este servicio está diseñado para:

- Ejecutarse automáticamente después de que se inicie la red (After=network.target).
- Lanzar el script start_server.sh, el cual realiza primero un git pull para asegurarse de tener la última versión del código y después arranca el backend.

8. Dockerización de la aplicación

Para facilitar el despliegue y la gestión de IoTHarvest, se ha dockerizado toda la aplicación usando tres contenedores: uno para el frontend, otro para el backend, y un tercero para el modelo de predicción.

Cada componente se aísla en su contenedor correspondiente, lo que permite una mayor organización y mantenimiento del sistema.

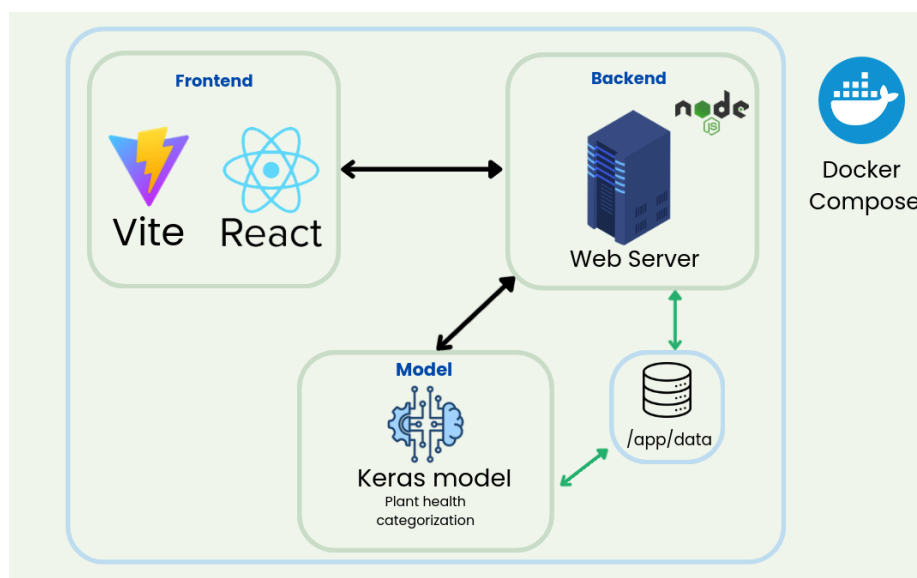
Dado que los contenedores son efímeros, se ha definido un volumen compartido entre el backend y el modelo, permitiendo que ambos servicios intercambien archivos (como las imágenes y los resultados de predicción) sin pérdida de datos entre reinicios o despliegues.

El archivo docker-compose.yml orquesta estos tres servicios, definiendo:

- Las dependencias entre contenedores.
- Los puertos expuestos
- El volumen compartido para mantener persistencia de datos entre el backend y el modelo.

Cada contenedor se construye a partir de un Dockerfile distinto:

- **Dockerfile.backend:** Prepara el entorno Node.js, instala las dependencias necesarias, copia el código fuente y expone el puerto 8080 para servir la API.
- **Dockerfile.frontend:** Compila la aplicación React, sirve los archivos estáticos en el puerto 8081 y proporciona acceso rápido a la interfaz web.
- **Dockerfile.model:** Construye el entorno Python necesario para el modelo de predicción, instala las bibliotecas de IA requeridas y ejecuta el script de análisis.



9. Potencial

Si bien la implementación actual cubre de manera integral los objetivos propuestos en la etapa inicial, se reconoce que el proyecto posee un alto potencial de escalabilidad. La solución desarrollada permite su expansión tanto en funcionalidad como en alcance, abriendo la posibilidad de incorporar mejoras y nuevas líneas de investigación.

A continuación, se presentan posibles direcciones en las que podrían enfocarse futuras investigaciones, organizadas en función de las principales secciones del proyecto. Estas proyecciones permiten visibilizar el margen de crecimiento del trabajo realizado y su adaptabilidad a contextos más amplios o complejos.

9.1 Hardware huerto

Una de las principales vías de escalabilidad del proyecto se encuentra en la posibilidad de ampliar el número de sensores desplegados en el terreno, con la idea de monitorizar cultivos en terrenos mucho más extensos.

Actualmente los sensores se comunican con el gateway mediante LoRa, una solución eficiente y de bajo consumo energético que ha permitido establecer una red fiable entre los sensores y la unidad central dentro de un área geográfica limitada.

No obstante, con vistas a una posible expansión del sistema hacia terrenos más extensos y una mayor cantidad de nodos, se plantea como evolución natural la implementación de una infraestructura basada en LoRaWAN. Esta tecnología permitiría gestionar un mayor volumen de sensores de forma más escalable y centralizada, mediante el uso de gateways que amplían significativamente el alcance y robustez de la red.

La transición a LoRaWAN abriría además la posibilidad de integrar el sistema con plataformas en la nube, facilitando la recolección, almacenamiento y análisis de datos en tiempo real desde ubicaciones remotas. Esta mejora tecnológica representa un paso clave para adaptar el proyecto a contextos de mayor escala, como explotaciones agrícolas de grandes dimensiones o redes distribuidas de sensores ambientales.

9.2 Servidor web

Otro componente esencial para la escalabilidad y robustez del sistema es la incorporación de un servidor Nginx para exponer el frontend. Al situar Nginx como proxy inverso frente a los contenedores, se obtienen múltiples beneficios:

- **Balanceo de carga y proxy inverso:** permite distribuir el tráfico entre varios servicios o contenedores, evitando cuellos de botella y mejorando la disponibilidad.
- **Cacheo y compresión:** optimiza los tiempos de respuesta mediante la cache de recursos estáticos y la compresión de contenido, especialmente útil en dispositivos con conexiones limitadas.
- **Implementación de HTTPS:** mediante la configuración de certificados TLS/SSL, se garantiza la transmisión segura de los datos entre el cliente y el servidor, un requisito esencial conforme crece la base de usuarios y aumenta la sensibilidad de la información gestionada.

Esta arquitectura basada en Nginx + HTTPS aporta mayor fiabilidad, mejora el rendimiento y proporciona una base sólida para el crecimiento progresivo del sistema.

9.3 MongoDB

Hasta ahora, tal y como se ha comentado en la sección de la base de datos en la colección `sensorsDataOptimized`, nos aprovechamos de Time Series almacenando los datos recibidos en buckets configurados por intervalos temporales.

En cambio, si se pretende escalar el proyecto, habría que tener en cuenta que uno o varios usuarios podrían disponer de más de un cultivo. Esto permitiría darle juego al campo no utilizado `metaField`.

Este campo ofrece la posibilidad de incorporar información adicional no temporal que permite organizar e identificar los datos de una manera más eficiente. Con este campo, se podrían incluir identificadores como el id de un usuario, la ubicación de un cultivo de cierto usuario o el propio id de un cultivo, tal y como se ha mencionado antes. Esto facilitaría la segmentación y el filtrado eficiente de los datos de una misma colección, permitiendo optimizar las consultas, mejorar el rendimiento en el acceso a la información y evitar la necesidad de crear una colección independiente para cada cultivo o usuario. Esto contribuiría a una arquitectura de datos más escalable, centralizada y mantenible.

9.4 Frontend

De cara a una posible expansión futura del proyecto, el frontend ofrece un gran potencial para evolucionar hacia una interfaz más robusta y profesional. Una opción interesante sería integrar Tailwind CSS y shadcn/ui, que permitirían escalar el diseño de manera ordenada y coherente, facilitando la incorporación de nuevas vistas o funcionalidades. Tailwind CSS aportaría una forma rápida y flexible de aplicar estilos personalizados, mientras que shadcn/ui proporcionaría una base sólida de componentes reutilizables y accesibles, ideal para construir interfaces más complejas sin perder simplicidad. Estas herramientas permitirían mantener una experiencia de usuario clara y fluida a medida que el sistema crezca.

9.5 Modelo IA

Como se mencionó en la presentación final, el modelo de IA se puede escalar muchísimo. Primeramente utilizando un dataset más parecido se podrían conseguir predicciones mucho más realistas.

También se podría añadir que no fuera una clasificación binaria sino que clasificar distintos tipos de enfermedades a distintos tipos de plantas.

Para ello habría que entrenar con muchísimos datasets i muchísimas imágenes, probablemente con hardware que lo soportara, pero es completamente posible con un poco más de recursos y tiempo.

9.6 Docker Compose a Kubernetes

Pasar de Docker Compose a Kubernetes es el siguiente paso lógico cuando la aplicación crece y necesita una gestión más completa. Kubernetes permite:

- **Orquestación avanzada:** distribuye automáticamente los servicios entre distintos nodos, gestiona el escalado dinámico según la carga y balancea el tráfico de forma eficiente.
- **Alta disponibilidad y tolerancia a fallos:** reinicia y redistribuye contenedores ante fallos, garantizando que los servicios permanezcan activos sin intervención manual.

- **Escalado horizontal:** ajusta el número de réplicas de cada servicio en función de la demanda, optimizando tanto el rendimiento como el uso de recursos.
- **Gestión centralizada de configuraciones y secretos:** permite una administración segura y controlada de variables de entorno, certificados y credenciales, facilitando el despliegue en diferentes entornos con altos estándares de seguridad.

Con Kubernetes, se abre la puerta a un despliegue mucho más flexible, robusto y automatizado, preparando la plataforma IoTHarvest para afrontar un aumento significativo de carga, usuarios o complejidad de requisitos en el futuro.

10. Código del proyecto

El código completo del proyecto está disponible en el siguiente repositorio de GitHub: <https://github.com/ermias-vm/loTHarvest>.

El repositorio está organizado en cinco carpetas principales: frontend, backend, scripts, model y hardware. Cada una incluye un archivo README.md donde se explica su contenido y funcionamiento.

Además, en la raíz del repositorio se encuentra una guía paso a paso para instalar las dependencias necesarias y poner en marcha la aplicación.

11. Links de interés

Links de compra de los componentes hardware utilizados para la implementación del sistema de monitorización del cultivo:

[1] TTGO LORA32 (*Sender*):

https://tienda.bricogeek.com/lora/1122-ttgo-lora32-esp32-con-oled-900-mhz.html?_gl=1*a9lgzc*_up*MQ..*_gs*MQ..&gclid=CjwKCAjwi-DBBhA5EiwAXOHsGdVW6J3cbMOxc4ZwU3VEqctCGmg45AllmofvGuxZDUa_JsoQ62GvkRoCYG0QAvD_BwE&gbraid=0AAAAADkb14c86wnl0QgB4eAh3WQnVyD-d

[2] TTGO T-Beam ESP32 (*Receiver*):

https://tienda.bricogeek.com/lora/1502-ttgo-t-beam-esp32-wifi-gps-neo-6m-lora-900-mhz.html?_gl=1*a9lgzc*_up*MQ..*_gs*MQ..&gclid=CjwKCAjwi-DBBhA5EiwAXOHsGdVW6J3cbMOxc4ZwU3VEqctCGmg45AllmofvGuxZDUa_JsoQ62GvkRoCYG0QAvD_BwE&gbraid=0AAAAADkb14c86wnl0QgB4eAh3WQnVyD-d

[3] ESP32-CAM OV2640:

https://tienda.bricogeek.com/arduino-compatibles/1912-esp32-cam-esp32-con-camera-ov2640.html?_gl=1*a9lgzc*_up*MQ..*_gs*MQ..&gclid=CjwKCAjwi-DBBhA5EiwAXOHsGdVW6J3cbMOxc4ZwU3VEqctCGmg45AllmofvGuxZDUa_JsoQ62GvkRoCYG0QAvD_BwE&gbraid=0AAAAADkb14c86wnl0QgB4eAh3WQnVyD-d

[4] Capacitive Soil Moisture Sensor:

https://tienda.bricogeek.com/sensores-humedad/1075-sensor-de-humedad-del-suelo-anticorrosion.html?_gl=1*a9lgzc*_up*MQ..*_gs*MQ..&gclid=CjwKCAjwi-DBBhA5EiwAXOHsGdVW6J3cbMOxc4ZwU3VEqctCGmg45AllmofvGuxZDUa_JsoQ62GvkRoCYG0QAvD_BwE&gbraid=0AAAAADkb14c86wnl0QgB4eAh3WQnVyD-d

[5] Sensor DHT11:

https://tienda.bricogeek.com/sensores-temperatura/1574-modulo-sensor-dht11-humedad-y-temperatura.html?_gl=1*a9lgzc*_up*MQ..*_gs*MQ..&gclid=CjwKCAjwi-DBBhA5EiwAXOHsGdVW6J3cbMOxc4ZwU3VEqctCGmg45AllmofvGuxZDUa_JsoQ62GvkRoCYG0QAvD_BwE&gbraid=0AAAAADkb14c86wnl0QgB4eAh3WQnVyD-d

Links de interés con documentación sobre las tecnologías usadas:

[6] Documentación oficial de Time Series:

<https://www.mongodb.com/docs/manual/core/timeseries-collections/>

Links de interés modelo de Keras:

[7] Dataset usado: Mendeley Data “Image Dataset for Radish Plant Leaf Disease Detection and Freshness Assessment from Bangladesh”

<https://data.mendeley.com/datasets/s973cz2jcd/1>

[8] Focal Loss vs. Binary Cross Entropy Loss - Dos maneras distintas para el entrenamiento del modelo.

<https://blog.dailydoseofds.com/p/focal-loss-vs-binary-cross-entropy>

[9] “DenseNet Explained” de Geeks for Geeks: Como funciona esta CNN y porque decidimos cambiarlo a este. <https://www.geeksforgeeks.org/densenet-explained/>